SOFTWARE ENGINEERING CAPSTONE DOCUMENTATION

BizAI Enterprise Platform

System Implementation & Technical Architecture Manual

A Comprehensive Engineering Guide to Layered Architecture, Atomic Stock Reservations, Idempotent Billing, and Context-Grounded AI Inference

---

Standard Compliance:	**Ian Sommerville, Software Engineering (10th Ed.) & UML 2.5 (ISO/IEC 19505)**
Project Architecture:	**Strict 4-Layer (React 19 SPA • FastAPI ASGI • SQLAlchemy ORM • SQLite/PostgreSQL)**
Engineering Team:	**Team YATRI**: Mohammad Ibad Hussain (Lead), Yash Narang, Ashutosh Yadav, Tushar, Ramji
Target Codebase:	**backend/** (FastAPI, SQLAlchemy, PyJWT, ReportLab) + **frontend/** (React 19, Vite)
Inference Stack:	**Meta LLaMA 3.1 8B** on Groq Cloud LPUs with deterministic JSON snapshot grounding
Document Status:	**Baselined Implementation Report (Version 1.0) • Academic Term 2026**

Executive Implementation Summary: BizAI is an enterprise-grade ERP platform addressing operational fragmentation in small-to-medium retail and wholesale enterprises. It integrates Customer Relationship Management, Dynamic Catalog Inventory, Two-Phase Sales Order Orchestration, Vector PDF Invoicing, Operating Expense Auditing, and Context-Grounded AI Analytics. This document details the technical implementation, software patterns, concurrency safety mechanisms, database models, and REST contracts realized in the production repository.

1. System Architecture & Technology Stack

In accordance with **Ian Sommerville's Software Engineering (10th Ed., Chapter 6, §6.3.1)**, BizAI implements a **Strict Layered Architectural Style**. System functionality is organized into discrete horizontal layers, where each layer communicates exclusively with the layer directly beneath it through standardized interfaces.

Layer	Core Technology	Implementation Responsibility in BizAI
Presentation Tier	React 19 SPA • Vite • Tailwind • Chart.js 4	Zero-reload Single-Page Application; Axios HTTP client with JWT interceptors; responsive component views.
API & Security Tier	FastAPI • Starlette ASGI • Pydantic v2	11 APIRouters; OAuth2 Password Bearer; compile-time request validation; RoleChecker RBAC callables.
Domain Services	Python 3.10+ Business Modules	Pure business logic: two-phase order reservation, invoice generation, profitability formulas, snapshot builder.
Persistence Tier	SQLAlchemy 2.0 ORM • SQLite / PostgreSQL	Dialect-agnostic relational mapping; ACID transaction boundaries; connection pooling; relationship cascades.
External AI Tier	Groq LPU API • Meta LLaMA 3.1 8B	Sub-second inference grounded with deterministic in-memory business snapshot; local fallback strategy.

Key Architectural Decision: SQLite WAL vs. PostgreSQL

The Evaluation & Portability Strategy: The repository defaults to SQLite running with Write-Ahead Logging (PRAGMA journal_mode=WAL;). This guarantees zero-friction local execution for college evaluators without requiring external database daemons. However, because all queries use SQLAlchemy 2.0 ORM Declarative Base (backend/database.py), switching to production PostgreSQL requires altering only the single environment variable DATABASE_URL=postgresql://user:pass@host:5432/bizai, achieving 100% database engine portability.

2. Database Schema & Data Models Implementation

The relational schema comprises 10 normalized tables enforcing foreign key integrity, unique constraints, and cascade behaviors:

Table Name	Primary Key	Foreign Keys & Constraints	Business Purpose
`roles`	`id (Int)`	`name (UK)`	Pre-seeded roles: admin, manager, employee, accountant.
`users`	`id (Int)`	`role_id → roles.id, email (UK)`	Staff accounts with bcrypt-hashed credentials and active flags.
`customers`	`id (Int)`	`email (UK, nullable)`	Commercial customer directory, GSTIN, and credit ledger.
`categories`	`id (Int)`	`name (UK)`	Organizational classification for product catalog items.
`products`	`id (Int)`	`category_id → categories.id, sku (UK)`	Catalog items, cost price, retail price, stock, and low-stock threshold.
`orders`	`id (Int)`	`customer_id → customers.id, created_by`	Sales order records with status state machine and order totals.
`order_items`	`id (Int)`	`order_id → orders.id (Cascade), product_id`	Line items with purchased quantity, historical unit price, and subtotal.
`invoices`	`id (Int)`	`order_id → orders.id (UK), invoice_number (UK)`	Billing records, balance due, payment status, and due dates.
`expenses`	`id (Int)`	`created_by → users.id`	Operational expenditures categorized for net profit auditing.
`inventory_movements`	`id (Int)`	`product_id → products.id, created_by`	Immutable audit log tracking all quantity additions and deductions.

3. Backend Subsystems & Business Logic Realization

3.1 Authentication & Role-Based Access Control (RBAC)

Authentication is stateless, utilizing **OAuth2 Password Bearer flow** and **PyJWT** (backend/routers/auth.py). User passwords undergo 12-round salted hashing using passlib[bcrypt]. Role-Based Access Control is enforced strictly at the API dependency layer via the RoleChecker callable class in backend/utils/permissions.py:

```
# backend/utils/permissions.py - Server-Side Role Enforcement Guard class RoleChecker: def
__init__(self, allowed_roles: list[str]): self.allowed_roles = allowed_roles def __call__(self, user:
User = Depends(get_current_active_user)): if user.role.name not in self.allowed_roles: raise
HTTPException(status_code=403, detail='Operation not permitted') return user require_manager =
RoleChecker(['admin', 'manager']) require_accountant = RoleChecker(['admin', 'manager',
'accountant'])
```

3.2 Dynamic Stock Reservation & Order State Machine

In backend/services/order_service.py, orders implement a formal two-phase state machine that guarantees inventory integrity under concurrent access. Stock is **not** reserved when an order is created as a draft (pending). Upon confirmation, stock availability is verified across all items; if any item is insufficient, a ValueError triggers an atomic database ROLLBACK. If confirmed orders are later cancelled, deducted stock is automatically restored:

```
# backend/services/order_service.py - Atomic Stock Reservation and Rollback if new_status ==
'confirmed' and current_status == 'pending': for item in order.items: # update_stock raises
ValueError if current stock < requested quantity product_service.update_stock(db, item.product_id,
-item.quantity, commit=False) # Stage immutable inventory movement log movement =
InventoryMovement(product_id=item.product_id, quantity_change=-item.quantity, reason='Order
Confirmation', created_by=current_user.id) db.add(movement) order.status = 'confirmed' db.commit() #
Commits order update, stock decrement, and movement logs atomically
```

3.3 Programmatic Vector PDF Invoicing Engine

Invoice creation is idempotent: calling POST /api/invoices/from-order/{id} creates a new invoice with status sent and a 15-day due date, or returns the existing invoice if already generated. Server-side PDF generation is executed programmatically via ReportLab (backend/utils/pdf.py), generating vector-quality PDF buffers in RAM and returning them as streaming HTTP responses (media_type='application/pdf').

3.4 Operating Expense Auditing & Real-Time Profitability Formula

Financial intelligence is computed dynamically in backend/services/report_service.py without maintaining stale cache columns:
- **Gross Sales Revenue:** sum(Order.total_amount) WHERE status != 'cancelled'
- **Cost of Goods Sold (COGS):** sum(OrderItem.quantity * Product.cost_price) WHERE Order.status != 'cancelled'
- **Operating Overheads:** sum(Expense.amount)
- **Net Profit: Total Sales Revenue** – **COGS** – **Operating Overheads**

4. Context-Grounded AI Assistant Architecture

BizAI eliminates LLM hallucinations through a **Deterministic Context-Grounded Architecture** (Sommerville SE10, §7.3). The AI assistant never generates or executes arbitrary SQL queries, preventing SQL injection vulnerabilities. Instead, it implements the Strategy Pattern across three distinct phases:

Phase	Subsystem Execution	Technical Integrity Safeguard
1. Snapshot Assembly	report_service.build_business_snapshot(db)	Executes deterministic SQLAlchemy aggregations extracting revenue, net profit, low-stock items, top products, and unpaid invoices into a clean JSON dictionary.
2. Prompt Grounding	Injected into Meta LLaMA 3.1 8B via Groq LPU SDK	System prompt strictly mandates: *'Answer ONLY using the provided JSON snapshot. Never invent or hallucinate metrics.'* Inference completes in <1.2 seconds.
3. Fallback Degradation	_fallback_answer() Deterministic Engine	If the Groq API key is missing or network connectivity fails, internal keyword analysis answers queries directly from the snapshot without crashing.

5. Frontend Single-Page Application (SPA) Architecture

The user interface is built on **React 19** and bundled using **Vite**. The client maintains zero full-page reloads, achieving desktop-like performance via the following architectural components:

Module / Layer	File Path in Codebase	Architectural Realization
Router Controller	frontend/src/App.jsx	Defines 13 protected and public routes using React Router 7. Enforces layout wrappers.
Auth Context Store	frontend/src/context/AuthContext.jsx	Maintains active user token in localStorage; decodes claims and broadcasts auth state to components.
Axios HTTP Client	frontend/src/api/client.js	Configured with request interceptors injecting Bearer <token>; response interceptors redirect 401s to /login.
Interactive Charts	frontend/src/pages/Reports.jsx	Chart.js 4 canvas instances rendering 30-day sales series, margin distributions, and product breakdowns.
Component Library	frontend/src/components/	Reusable DataTable with live search filters, StatusBadge, Modal dialogs, and confirm prompts.

6. Complete REST API Specification & Security Contracts

BizAI exposes 24 high-throughput REST endpoints. All routes require valid Bearer JWTs except /api/auth/login:

HTTP	Route Path	RBAC Permission	Status	Functional Responsibility
POST	/api/auth/login	Public	200	Authenticates credentials; returns 30-min HMAC-SHA256 JWT.
GET	/api/auth/me	require_staff	200	Returns current authenticated user profile and permissions.
GET	/api/users	require_admin	200	Lists all provisioned staff accounts.
POST	/api/users	require_admin	201	Provisions new user with bcrypt-hashed credentials.
GET	/api/customers	require_staff	200	Paginated, searchable commercial customer directory.
POST	/api/customers	require_employee	201	Registers customer profile with contact & GSTIN numbers.
GET	/api/products	require_staff	200	Product master catalog with real-time stock levels.
POST	/api/products	require_manager	201	Creates catalog item with cost price and low-stock alert threshold.
POST	/api/products/{id}/stock	require_manager	200	Manual stock adjustment override; writes immutable movement row.
GET	/api/products/low-stock	require_staff	200	Filters products where stock_qty ≤ min_stock_level.
GET	/api/products/history	require_staff	200	Complete inventory movement audit log with timestamps & users.
GET	/api/orders	require_staff	200	Lists sales orders with embedded customer summary and line items.
POST	/api/orders	require_employee	201	Composes draft sales order in 'pending' status.
PATCH	/api/orders/{id}/status	require_manager	200	State transition; triggers atomic stock deduction on confirmation.
GET	/api/invoices	require_staff	200	Queries enterprise invoices, balance due, and payment states.
POST	/api/invoices/from-order/{id}	require_staff	201	Idempotent invoice generator; sets 15-day due date.
PATCH	/api/invoices/{id}/status	require_accountant	200	Records partial/full payment, updates balance due, settles invoice.
GET	/api/invoices/{id}/pdf	require_staff	200	Server-side vector PDF invoice compilation via ReportLab.
GET	/api/expenses	require_staff	200	Queries categorized operational expense records.
POST	/api/expenses	require_accountant	201	Logs operational expenditure with receipt description.
GET	/api/reports/dashboard	require_staff	200	Executive metrics: Total Revenue, COGS, Net Profit, Low Stock.
POST	/api/ai/chat	require_staff	200	Constructs live JSON snapshot; queries LLaMA 3.1 8B or fallback.
POST	/api/demo/generate	require_admin	200	Seeds database with 20 customers, 50 products, 100 orders.
DELETE	/api/demo/clear	require_admin	200	Purges transactional tables for clean presentation resets.

7. Software Design Patterns Realized in Code

In accordance with **Ian Sommerville SE10 Chapter 7 (§7.3)**, BizAI explicitly implements four recognized design patterns:

Design Pattern	Implementation Location in Repository	Architectural Rationale & Behavior
Strategy Pattern	backend/services/ai_service.py	Decouples AI inference provider from business consumers. The system dynamically selects between cloud Groq LPUs (Meta LLaMA 3.1 8B) and local keyword-matching deterministic fallback.
Facade Pattern	backend/services/report_service.py	build_business_snapshot() provides a simple, unified interface over complex relational aggregations across orders, products, inventory movements, and expenses.
Dependency Injection	FastAPI Depends() in all routers	Injects database sessions (get_db), authenticated users (get_current_active_user), and authorization guards (RoleChecker) cleanly without hardcoding dependencies.
Repository Pattern	backend/services/*.py modules	Isolates raw SQLAlchemy ORM model queries within dedicated service functions, preventing HTTP controllers from coupling directly to persistence mechanisms.

8. Verification, Testing & Deployment

Verification & Validation (Sommerville SE10 Ch. 8): System correctness was verified across three levels:
1. **Automated Pydantic Compile-Time Validation:** Request payloads are checked before handler execution. Invalid types, missing emails, or negative stock quantities return HTTP 422 immediately.
2. **Transactional Integrity Test Cases:** Simulated order confirmation with insufficient inventory verifies that the database state rolls back cleanly without partial stock deduction.
3. **End-to-End Sandbox Simulation:** The built-in demo engine (/api/demo/generate) populates 20 customers, 50 products, 100 orders, and 30 expenses, validating all aggregation formulas under load.
Cloud Deployment: Configured via render.yaml for automated cloud deployment, supporting Gunicorn/Uvicorn workers on Linux hosts with PostgreSQL add-ons.